Advanced Rust

alongside why rust is hard to write and why that’s good

not feedback, rust is kind of annoying to code.

I think the Rust lectures in class are a little scatterbrained and hard to follow at times. I would like to see more cohesion and justification for the various choices Rust makes. It is easy enough to look up something in the Rust book - class should focus on the *why*, as well as the *how* of how to write idiomatic Rust.

What is idiomatic Rust? How do I write it?

What is idiomatic Rust? How do I write it?

What is Rust's design philosophy?

Wikipedia
https://en.wikipedia.org › wiki › Rust_(programming_l...

Rust (programming language)

Rust is a general-purpose programming language. It is noted for its emphasis on performance, type safety, concurrency, and memory safety.

History | Syntax and features

Wikipedia
https://en.wikipedia.org › wiki › Rust

Rust

Rust is **an iron oxide**, a usually reddish-brown oxide formed by the reaction of iron and oxygen in the catalytic presence of water or air moisture.

Wikipedia
https://en.wikipedia.org › wiki › Rust_(video_game)

Rust (video game)

Rust is a multiplayer survival video game developed by Facepunch Studios. It was first released in early access in December 2013 and received its full release ...

Wikipedia
https://en.wikipedia.org › wiki › Rust_(2024_film)

Rust (2024 film)

Rust is a 2024 **American Western film** written and directed by Joel Souza. It stars Alec Baldwin (who also produced and co-wrote the story with Souza)

Rust is a general-purpose programming language. It is noted for its emphasis on performance, type safety, concurrency, and memory safety.

Rust supports multiple programming paradigms. It was influenced by ideas from functional programming, including immutability, higher-order functions, algebraic data types, and pattern matching. It also supports object-oriented programming via structs, enums, traits, and methods. Rust is noted for enforcing memory safety (i.e., that all references point to valid memory) without a conventional garbage collector; instead, memory safety errors and data races are prevented by the "borrow checker", which tracks the object lifetime of references at compile time.

Rust is a general-purpose programming language. It is noted for its emphasis on performance, type safety, concurrency, and memory safety.

Rust supports multiple programming paradigms. **It was influenced by ideas from functional programming, including immutability, higher-order functions, algebraic data types, and pattern matching.** It also supports object-oriented programming via structs, enums, traits, and methods. Rust is noted for enforcing memory safety (i.e., that all references point to valid memory) without a conventional garbage collector; instead, memory safety errors and data races are prevented by the "borrow checker", which tracks the object lifetime of references at compile time.

Rust is a general-purpose programming language. It is noted for its emphasis on performance, type safety, concurrency, and memory safety.

Rust supports multiple programming paradigms. **It was influenced by ideas from functional programming, including immutability, higher-order functions, algebraic data types, and pattern matching.** It also supports **object-oriented programming via structs, enums, traits, and methods.** Rust is noted for enforcing memory safety (i.e., that all references point to valid memory) without a conventional garbage collector; instead, memory safety errors and data races are prevented by the "borrow checker", which tracks the object lifetime of references at compile time.

Yes, but this is more of a laundry list of features

Rust is a g age. It is noted for its emphasis on performanc emory safety.

Rust supports multiple programming paradigms. **It was influenced by ideas from functional programming, including immutability, higher-order functions, algebraic data types, and pattern matching.** It also supports **object-oriented programming via structs, enums, traits, and methods.** Rust is noted for enforcing memory safety (i.e., that all references point to valid memory) without a conventional garbage collector; instead, memory safety errors and data ra necker", which tracks the object lifetime of referenc

Kind of, but Rust is better suited to other paradigms…

My* Answer: Type Driven Design

The goals for this lecture are:

1. Recognize what TDD can do
   a. Answer how and why this form of design is useful
2. Show the potential of Rust's type system in supporting TDD
3. Show that TDD is integral to even Rust's internals and overall design
   a. Understand a little more about `unsafe`, `async`/`await`, and `Pin`

* I obviously did not come up with this myself, but it's the answer I would offer.

Thinking about Lists

Let's say we have some list of integers, `my_list: List<u32>`

```
my_list.sum()
my_list.product()
my_list.double_all()
my_list.clear()
my_list.push()
```

```
my_list.first()
my_list.average()
my_list.min()
```

Thinking about Lists

Let's say we have some list of integers, `my_list: List<u32>`

```
my_list.sum()
my_list.product()
my_list.double_all()
my_list.clear()
my_list.push()
```

```
my_list.first()
my_list.average()
my_list.min()
```

Some functions are well defined for ***ALL*** lists, others are not!

Thinking about Lists: A More Concrete Example

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    pub name: String,
    pub role: Role
}
```

```
struct Team {
    pub members: Vec<Person>,
}
```

```rust
fn get_team_by_id(id: &Id) -> Option<Team> { ... }

/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Vec<&Person> {
    team.members
        .iter()
        .filter(|person| !matches!(person.role, Role::JuniorDev))
        .collect()
}

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&id);
    if let Some(team) = team {
        let rotation = create_on_call_rotation(&team);
    }
}
```

```rust
fn get_team_by_id(id: &Id) -> Option<Team> { ... }

/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Vec<&Person> {
    team.members
        .iter()
        .filter(|person| !matches!(person.role, Role::JuniorDev))
        .collect()
}

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&id);
    if let Some(team) = team {
        let rotation = create_on_call_rotation(&team);
    }
}
```

Wait, what if members is empty?

```rust
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Result<Vec<&Person>, ()> {
    if team.members.is_empty() {
        Err(())
    } else {
        Ok(team
            .members
            .iter()
            .filter(|person| !matches!(person.role, Role::JuniorDev))
            .collect())
    }
}

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&id);
    if let Some(team) = team {
        let rotation = create_on_call_rotation(&team);
    }
}
```

```rust
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Result<Vec<&Person>, TeamError> { ... }

/// Point of contact should be team's PM.
fn get_point_of_contact(team: &Team) -> Result<&Person, TeamError> {
    team.members
        .iter()
        .find(|person| matches!(person.role, Role::PM))
        .ok_or(TeamError::MissingPm)
}

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&id);
    if let Some(team) = team {
        let rotation = create_on_call_rotation(&team);
        let poc = get_point_of_contact(&team);
    }
}
```

Again, we insert error handling

```rust
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Result<Vec<&Person>, TeamError> { ... }

/// Point of contact should be team's PM.
fn get_point_of_contact(team: &Team) -> Result<&Person, TeamError> { ... }

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&id);
    if let Some(team) = team {
        let rotation = create_on_call_rotation(&team);
        let poc = get_point_of_contact(&team);

        if let (Ok(rot), Ok(poc)) = (rotation, poc) {
            // email the poc with the rotation!
        } else {
            panic!("We are missing something!!!");
        }
    }
}
```

```rust
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Result<Vec<&Person>, TeamError> { ... }

/// Point of contact should be team's PM.
fn get_point_of_contact(team: &Team) -> Result<&Person, TeamError> { ... }

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&i
    if let Some(team) = team {
        let rotation = create_or
        let poc = get_point_of_c

        if let (Ok(rot), Ok(poc)) = (rotation, poc) {
            // email the poc with the rotation!
        } else {
            panic!("We are missing something!!!");
        }
    }
}
```

These functions assume nothing about the team, and therefore report all possible errors!

```rust
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Result<Vec<&Person>, TeamError> { ... }

/// Point of contact should be team's PM.
fn get_point_of_contact(team: &Team) -> Result<&Person, TeamError> { ... }

fn main() {
    let id = 123456
    let team = get_
    if let Some(tea
        let rotation = create_on_call_rotation(&team);
        let poc = get_point_of_contact(&team);

        if let (Ok(rot), Ok(poc)) = (rotation, poc) {
            // email the poc with the rotation!
        } else {
            panic!("We are missing something!!!");
        }
    }
}
```

Main has to assume any function call using `Team` could have failed!

Thinking about Lists: A More Concrete Example

The last few slides show one possible data model.

- Developers of the “team library”:
  - Assume nothing about the input data
  - Perform validations on each potentially-unsafe access
  - Relay all errors back to the caller
- Consumers of the “team library”:
  - Have to be mindful of where they get their data from, could require further validation
  - Have to handle every single possible error for every single function call

“Assume nothing” is a safe attitude, but it is simply not practical.

Parse, don't validate

Blog post by Alexis King

2019-11-05 functional programming, haskell, types

Historically, I've struggled to find a concise, simple way to explain what it means to practice type-driven design. Too often, when someone asks me "How did you come up with this approach?" I find I can't give them a satisfying answer. I know it didn't just come to me in a vision—I have an iterative design process that doesn't require plucking the "right" approach out of thin air—yet I haven't been very successful in communicating that process to others.

However, about a month ago, I was reflecting on Twitter about the differences I experienced parsing JSON in statically- and dynamically-typed languages, and finally, I realized what I was looking for. Now I have a single, snappy slogan that encapsulates what type-driven design means to me, and better yet, it's only three words long:

Parse, don't validate.

Types are Invariants

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    pub name: String,
    pub role: Role
}
```

```
struct Team {
    pub members: Vec<Person>,
}
```

Types are Invariants

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    pub name: String,
    pub role: Role
}
```

```
struct Team {
    pub members: Vec<Person>,
}
```

Nothing about these data definitions captures the above fact!

Types are Invariants

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    pub name: String,
    pub role: Role
}
```

```
struct Team {
    pub members: Vec<Person>,
}
```

***Impossible* states are *possible* to represent!**

Types are Invariants: Slightly Better…

Teams have at least one person on them (the PM), but could have more

```rust
struct NonEmptyVec<T> {
    inner: Vec<T>,
}

impl<T> NonEmptyVec<T> {
    pub fn new(v: Vec<T>) -> Self {
        if v.is_empty() { panic!("Cannot construct NonEmptyVec"); }
        Self { inner: v }
    }

    pub fn iter(&self) -> impl Iterator<Item = &T> {
        self.inner.iter()
    }
}
```

Types are Invariants: Slightly Better…

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    pub name: String,
    pub role: Role
}
```

```
struct Team {
    pub members: NonEmptyVec<Person>,
}
```

Types are Invariants: Slightly Better…

Teams have at least one person on them (the PM), but could have more

```rust
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Result<Vec<&Person>, TeamError> {
    if team.members.is_empty() {
        Err(TeamError::MissingMembers)
    } else {
        Ok(team
            .members
            .iter()
            .filter(|person| !matches!(person.role, Role::JuniorDev))
            .collect())
    }
}
```

Types are Invariants: Slightly Better…

Teams have at least one person on them (the PM), but could have more

```rust
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Result<Vec<&Person>, TeamError> {
    if team.members.is_empty() {
        Err(TeamError::MissingMembers)
    } else {
        Ok(team
            .members
            .iter()
            .filter(|person| !matches!(person.role, Role::JuniorDev))
            .collect())
    }
}
```

We don't need this check!

Types are Invariants: Slightly Better…

Teams have at least one person on them (the PM), but could have more

```
/// Junior Devs don't need to participate in on-call!
fn create_on_call_rotation(team: &Team) -> Vec<&Person> {
    team
        .members
        .iter()
        .filter(|person| !matches!(person.role, Role::JuniorDev))
        .collect()
}
```

Types are Invariants: Actually Better

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    pub name: String,
    pub role: Role
}
```

```
struct Team {
    pub members: NonEmptyVec<Person>,
}
```

Types are Invariants: Actually Better

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    pub name: String,
    pub role: Role
}
```

```
struct Team {
    pub members: NonEmptyVec<Person>,
}
```

*We still have **potential** for impossible state!*

Types are Invariants: Actually Better

Teams have at least one person on them (the PM), but could have more

```
enum Role {
    JuniorDev,
    SeniorDev,
    PM,
}
```

```
struct Person {
    name: String,
    role: Role
}
```

```
struct Team {
    lead: Person,
    members: Vec<Person>,
}
```

<u>New Invariant:</u>

`team.lead` *is always a PM, and always defined for all teams*

Types are Invariants: Actually Better

Teams have at least one person on them (the PM), but could have more

```rust
struct Team {
    lead: Person,
    members: Vec<Person>,
}
```

```rust
impl Team {
    pub fn new(lead: Person, members: Vec<Person>) -> Option<Self> {
        if !matches!(lead.role, Role::PM) {
            None
        }

        Some(Self { lead, members })
    }
}
```

```rust
fn create_on_call_rotation(team: &Team) -> Vec<&Person> {
    let mut res = vec![&team.lead];
    res.append(&mut team.members.iter()
                    .filter(|person| !matches!(person.role, Role::JuniorDev))
                    .collect());
    res
}

fn get_point_of_contact(team: &Team) -> &Person {
    &team.lead
}

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&id);
    if let Some(team) = team {
        let rotation = create_on_call_rotation(&team);
        let poc = get_point_of_contact(&team);
        // rotation and poc are guaranteed to be valid!
    }
}
```

```rust
fn create_on_call_rotation(team: &Team) -> Vec<&Person> {
    let mut res = vec![&team.lead];
    res.append(&mut team.members.iter()
                    .filter(|person| !matches!(person.role, Role::JuniorDev))
                    .collect());
    res
}

fn get_point_of_contact(team: &Team) -> &Person {
    &team.lead
}

fn main() {
    let id = 12345678;
    let team = get_team_by_id(&id);
    if let Some(team) = team {
        let rotation = create_on_call_rotation(&team);
        let poc = get_point_of_contact(&team);
        // rotation and poc are guaranteed to be valid!
    }
}
```

No unnecessary validation!

No need to check for errors!

Thinking about Lists: A More Concrete Example

The last few slides show a better data model!

- Developers of the “team library”:
  - Get to use assumptions about the data they are working with
  - No longer need to handle impossible states
  - Benefit from separation of concerns
- Consumers of the “team library”:
  - Get to use assumptions about the data they are working with

An expressive type system allows for impossible states to be unrepresentable

- Types become contracts between the data’s creator and the data’s user

Parse, Don’t Validate in Rust

“That’s obvious! Of course you would do that in this toy example! Besides, this seems like general advice for all programming languages, what does this have to do with Rust in particular?”

– you, the unconvinced skeptic

Parse, Don’t Validate in Rust

“That’s obvious! Of course you would do that in this toy example! Besides, this seems like general advice for all programming languages, what does this have to do with Rust in particular?”

– you, the unconvinced skeptic

- Yes, this is good advice for all programming languages
- The principle of using invariants as you write Rust shows up **absolutely everywhere**

Invariants in Rust

`rustc` exists to prove different invariants.

A Brief Discussion on Invariants in Rust: Essential Properties to Uphold

2023-08-15 TOETOE55

Aug 22, 2018 • Internship, Rust • Edits • Permalink

Two Kinds of Invariants: Safety and Validity

When talking about the Rust type system in the context of unsafe code, the discussion often revolves around *invariants*: Properties that must always hold, because the language generally assumes that they do. In fact, an important part of the mission of the Unsafe Code Guidelines strike force is to deepen our understanding of what these invariants are.

However, in my view, *there is also more than one invariant*, matching the fact that there are (at least) two distinct parties relying on these invariants: The compiler, and (authors of) safely usable code. This came up often enough in recent discussions that I think it is worth writing it down properly once, so I can just link here in the future. :)

...variant" is never far behind. In the

...need to be upheld. For instance:

`true` or `false`.

Invariants in Rust

`rustc` exists to prove different invariants.

r/rust • 1y ago
horsetreefence

Good invariants to keep in mind while learning rust

Aug 22, 201

Two Kinds of Invariants: Sa

When talking about the Rust type system in the context of u

worriedjacket • 1y ago

invariants go in the type system not in your brain

27

or `false`.

Rust Compiler Development Guide

Invariants of the type system

FIXME: This file talks about invariants of the type system as a whole, not only the solver

Invariants in Rust

rustc e

Go

Two K

When talking

Invari

FIXME: This file talks about invariants of the type system as a whole, not only the solver

Jan 31, 2018 • Research, Rust • Edits • Permalink

Sharing for a Lifetime

This post is about an aspect of the RustBelt paper. Concretely, I'd like to share some of our thoughts on the nature of types and shared references. Let's see how well this goes. :)

Shared references are an extremely powerful mechanism in the Rust type system, and we've had quite some trouble finding a good way of handling them in our formal model. In this post, I will present the model that we came up with. Let's get started by discussing what a "model" of a type looks like, and what's so hard about handling shared references.

Immutable Sharing

The reason shared references are so subtle is interior mutability. This fundamentally changes the nature of shared references. To see how, I will briefly assume that Rust does *not* permit interior mutability, and describe how a model of Rust types could roughly look like.

What's in a Type?

I've already written about the idea of "semantic types", which essentially boils down to the concept that types can impose *additional invariants* on their fields, like `self.cap >= self.len` (for `Vec`) or `!is_zero(self.0)` (for `NonZero`). So, whenever I say "(semantic) type", I really mean "a type's (syntactic) definition (aka the data layout) **and** the additional invariants that the module imposes on this type". Two types may be syntactically equal but have different invariants, and then clearly

rust

brain

For example, when you write a function `fn foo(x: bool)`, you can assume that `x` is safe for `bool`: It is either `true` or `false`. This means that every safe operation on `bool` (in particular, case distinction) can be performed without running into undefined behavior.

In fact, this is the main point of the safety invariant:

The safety invariant ensures that safe code, no matter what it does, executes safely – it cannot cause undefined behavior when working on this data.

…

When we talk about integer types like `i32`, it is tempting to say that all data is safe at `i32`. However, that does not actually work: As I discussed in a previous post, the "bytes" that make up data have more possible states than just `0..256`. In particular, they can be uninitialized (`undef` or `poison` in LLVM). Some operations like `if x != 0 { ... }` are undefined behavior if `x` is uninitialized, so sticking with our theme that the safety invariant must be strong enough to support safe execution of safe code, we have to declare that `x` is only safe at `i32` if it is initialized.

For reference types, some of the details are harder to define, but we can still roughly say that a pointer is safe for `&'a mut T` if it is aligned, non-NULL and points to allocated memory that, for lifetime `'a`, no other pointer accesses, and such that the data stored in memory at that location is safe for `T`. Similarly (and ignoring interior mutability), a pointer is safe for `&'a T` if it is aligned, non-NULL and points to allocated memory that is not mutated for lifetime `'a` and such that the data stored in memory is safe for `T`. The safety of `&bool` that we discussed above is just an instance of this general definition. In particular, it is impossible to have a safe `&!` because it would have to point to a safe `!`, which does not exist.

For example, ***when you write a function `fn foo(x: bool)`, you can assume that `x` is safe for `bool`****: It is either `true` or `false`. This means that every safe operation on `bool` (in particular, case distinction) can be performed without running into undefined behavior.*

In fact, this is the main point of the safety invariant:

The safety invariant ensures that safe code, no matter what it does, executes safely – it cannot cause undefined behavior when working on this data.

…

When we talk about integer types like `i32`, it is tempting to say that all data is safe at `i32`. However, that does not actually work: As I discussed in a previous post, the "bytes" that make up data have more possible states than just `0..256`. In particular, they can be uninitialized (`undef` or `poison` in LLVM). Some operations like `if x != 0 { ... }` are undefined behavior if `x` is uninitialized, so sticking with our theme that the safety invariant must be strong enough to support safe execution of safe code, we have to declare that `x` is only safe at `i32` if it is initialized.

For reference types, some of the details are harder to define, but we can still roughly say that a pointer is safe for `&'a mut T` if it is aligned, non-NULL and points to allocated memory that, for lifetime `'a`, no other pointer accesses, and such that the data stored in memory at that location is safe for `T`. Similarly (and ignoring interior mutability), a pointer is safe for `&'a T` if it is aligned, non-NULL and points to allocated memory that is not mutated for lifetime `'a` and such that the data stored in memory is safe for `T`. The safety of `&bool` that we discussed above is just an instance of this general definition. In particular, it is impossible to have a safe `&!` because it would have to point to a safe `!`, which does not exist.

For example, ***when you write a function `fn foo(x: bool)`, you can assume that x is safe for `bool`****: It is either `true` or `false`. This means that every safe operation on `bool` (in particular, case distinction) can be performed without running into undefined behavior.*

In fact, this is the main point of the safety invariant:

The safety invariant ensures that safe code, no matter what it does, executes safely – it cannot cause undefined behavior when working on this data.

…

When we talk about integer types like `i32`, it is tempting to say that all data is safe at `i32`. *However, that does not actually work: As I discussed in a previous post, the "bytes" that make up data have more possible states than just `0..256`. In particular, they can be uninitialized (`undef` or `poison` in LLVM).* ***Some operations like `if x != 0 { ... }` are undefined behavior if x is uninitialized****, so sticking with our theme that the safety invariant must be strong enough to support safe execution of safe code,* ***we have to declare that x is only safe at `i32` if it is initialized****.*

For reference types, some of the details are harder to define, but we can still roughly say that a pointer is safe for `&'a mut T` if it is aligned, non-NULL and points to allocated memory that, for lifetime `'a`, no other pointer accesses, and such that the data stored in memory at that location is safe for `T`. Similarly (and ignoring interior mutability), a pointer is safe for `&'a T` if it is aligned, non-NULL and points to allocated memory that is not mutated for lifetime `'a` and such that the data stored in memory is safe for `T`. The safety of `&bool` that we discussed above is just an instance of this general definition. In particular, it is impossible to have a safe `&!` because it would have to point to a safe `!`, which does not exist.

For example, ***when you write a function `fn foo(x: bool)`, you can assume that x is safe for `bool`***: It is either `true` or `false`. This means that every safe operation on `bool` (in particular, case distinction) can be performed without running into undefined behavior.

In fact, this is the main point of the safety invariant:

The safety invariant ensures that safe code, no matter what it does, executes safely – it cannot cause undefined behavior when working on this data.

…

When we talk about integer types like `i32`, it is tempting to say that all data is safe at `i32`. However, that does not actually work: As I discussed in a previous post, the "bytes" that make up data have more possible states than just `0..256`. In particular, they can be uninitialized (`undef` or `poison` in LLVM). ***Some operations like `if x != 0 { ... }` are undefined behavior if x is uninitialized***, so sticking with our theme that the safety invariant must be strong enough to support safe execution of safe code, ***we have to declare that x is only safe at `i32` if it is initialized***.

For reference types, some of the details are harder to define, but we can still roughly say that ***a pointer is safe for `&'a mut T` if it is aligned, non-NULL and points to allocated memory that, for lifetime `'a`, no other pointer accesses, and such that the data stored in memory at that location is safe for T.*** Similarly (and ignoring interior mutability), a pointer is safe for `&'a T` if it is aligned, non-NULL and points to allocated memory that is not mutated for lifetime `'a` and such that the data stored in memory is safe for T. The safety of `&bool` that we discussed above is just an instance of this general definition. In particular, it is impossible to have a safe `&!` because it would have to point to a safe `!`, which does not exist.

```rust
fn foo<'a>(param: &'a mut [usize]) { ... }
```

```rust
int foo(long *param)                { ... }
```

Invariants in Rust

`rustc` exists to prove different invariants.

- Each variable carries with it invariants that determine how it can be used
  - These invariants are encoded by the variable's type

- Safety invariants protect us from undefined behavior
  - "a `u32` is always initialized before it is read"
  - "a `&mut` is the sole live reference allowed to change the underlying value"
  - "references must refer to non-null, aligned data, and must outlive the data's owner"
  - Etc…

- Rust is frustrating to write exactly for this reason
  - If the compiler can't prove ***all*** safety invariants, it will fail to compile your code

The Scary Unsafe Block

We've alluded to `unsafe`'s existence a few times over the past two months

The Rust Programming Language

Unsafe Rust

All the code we've discussed so far has had Rust's memory safety guarantees enforced at compile time. However, Rust has a second language hidden inside it that doesn't enforce these memory safety guarantees: It's called *unsafe Rust* and works just like regular Rust but gives us extra superpowers.

Unsafe Rust exists because, by nature, static analysis is conservative. When the compiler tries to determine whether or not code upholds the guarantees, it's better for it to reject some valid programs than to accept some invalid programs. Although the code *might* be okay, if the Rust compiler doesn't have enough information to be confident, it will reject the code. In these cases, you can use unsafe code to tell the compiler, "Trust me, I know what I'm doing." Be warned, however, that you use unsafe Rust at your own risk: If you use unsafe code incorrectly, problems can occur due to memory unsafety, such as null pointer dereferencing.

The Scary Unsafe Block

We've alluded to `unsafe`'s existence a few times over the past two months

The Rust Programming Language

Performing Unsafe Superpowers

To switch to unsafe Rust, use the `unsafe` keyword and then start a new block that holds the unsafe code. You can take five actions in unsafe Rust that you can't in safe Rust, which we call *unsafe superpowers*. Those superpowers include the ability to:

1. Dereference a raw pointer.
2. Call an unsafe function or method.
3. Access or modify a mutable static variable.
4. Implement an unsafe trait.
5. Access fields of `union`s.

that you use unsafe Rust at your own risk. If you use unsafe code incorrectly, problems can occur

due to memory unsafety, such as null pointer dereferencing.

The Scary Unsafe Block

We've alluded to `unsafe`'s existence a few times over the past two months

> *For unsafe code authors, the slogan summarizing this post is:*
>
> *You must always be valid, but you only must be safe in safe code.*
>
> *– Ralf Jung*

The Scary Unsafe Block

We've alluded to `unsafe`'s existence a few times over the past two months

```
fn main() {
    let mut x = 10;

    let x_ptr1: *const i32 = &raw const x;
    let x_ptr2: *mut i32   = &raw mut x;
    let random = 0x12345678 as *mut i32;

    // Prints something like:
    // 0x7fff4f498664, 0x7fff4f498664
    println!("{x_ptr1:?}, {x_ptr2:?}");
}
```

The Scary Unsafe Block

We've alluded to `unsafe`'s existence a few times over the past two months

```
fn main() {
    let mut x = 10;

    let x_ptr1: *const i32 = &raw const x;
    let x_ptr2: *mut i32   = &raw mut x;
    let random = 0x12345678 as *mut i32;

    // Prints something like:
    // 0x7fff4f498664, 0x7fff4f498664
    println!("{x_ptr1:?}, {x_ptr2:?}");
}
```

I'm using raw pointers! Where's the unsafe block?

The Scary Unsafe Block

We've alluded to `unsafe`'s existence a few times over the past two months

```
fn main() {
    let mut x = 10;

    let x_ptr1: *const i32 = &raw const x;
    let x_ptr2: *mut i32   = &raw mut x;
    let random = 0x12345678 as *mut i32;

    // Prints something like:
    // 0x7fff4f498664, 0x7fff4f498664
    println!("{x_ptr1:?}, {x_ptr2:?}");
}
```

I'm using raw pointers! Where's the unsafe block?

A mutable and an immutable pointer at the same time?

The Scary Unsafe Block

```rust
fn main() {
    let mut x = 10;

    let x_ptr1: *const i32 = &raw const x;
    let x_ptr2: *mut i32   = &raw mut x;
    let random = 0x12345678 as *mut i32;

    let x_ptr1_contains = unsafe { *x_ptr1 };
    let x_ptr2_contains = unsafe { *x_ptr2 };

    // Prints something like:
    // 0x7fff4f498664 --> 10
    // 0x7fff4f498664 --> 10
    println!("{x_ptr1:?} --> {x_ptr1_contains:?}");
    println!("{x_ptr2:?} --> {x_ptr2_contains:?}");
}
```

```rust
fn takes_pointer(ptr: *mut i32) {
    unsafe { *ptr += 1 };
}

fn main() {
    let mut x = 10;
    let x_ptr: *mut i32 = &raw mut x;

    // Prints something like:
    // 0x7fff4f498664 --> 10
    // 0x7fff4f498664 --> 11
    println!("{x_ptr:?} --> {x:?}");
    takes_pointer(x_ptr);
    println!("{x_ptr:?} --> {x:?}");
}
```

Where Do We Want Unsafe?

Let's say we want to split a mutable reference to a slice into two…

```
fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();

    assert!(mid <= len);

    (&mut values[..mid], &mut values[mid..])
}
```

Where Do We Want Unsafe?

Let's say we want to split a mutable reference to a slice into two…

```
fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();

    assert!(mid <= len);

    (&mut values[..mid], &mut values[mid..])
}
```

Taking two mutable references!

This won't compile, even though the actual operation is memory safe!

- Returned slices don't overlap after all!

Where Do We Want Unsafe?

Let's say we want to split a mutable reference to a slice into two…

```
fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();
    let ptr: *mut i32 = values.as_mut_ptr();

    assert!(mid <= len);

    unsafe {
        (
            std::slice::from_raw_parts_mut(ptr, mid),
            std::slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}
```

Where Do We

Let's say we want t

```
fn split_at_mu
    let len = y
    let ptr: *m

    assert!(mid

    unsafe {
        (
            sto
            sto
        )
    }
}
```

std::slice

Function from_raw_parts_mut

Since 1.0.0 (const: 1.83.0) · Source

Search Settings Help Summary

```
pub const unsafe fn from_raw_parts_mut<'a, T>(
    data: *mut T,
    len: usize,
) -> &'a mut [T]
```

Performs the same functionality as `from_raw_parts`, except tha

To use this function, the caller has to guarantee these conditions are met.

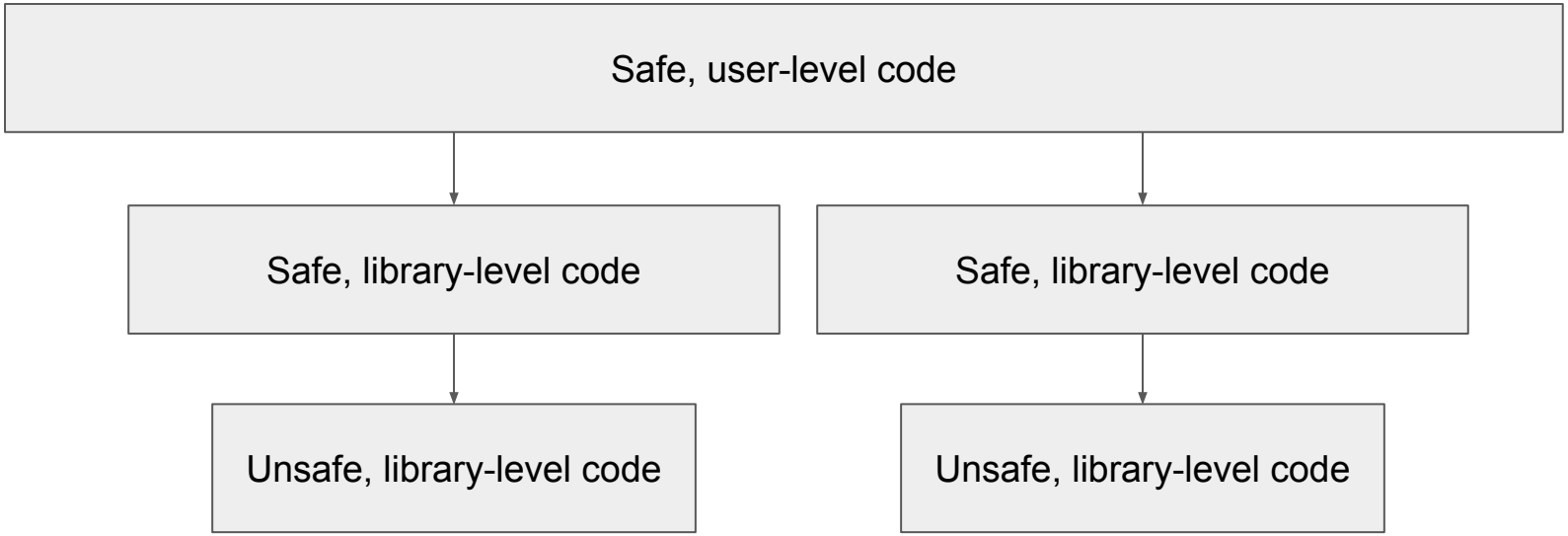
Safety

Behavior is undefined if any of the following conditions are violated:

- `data` must be non-null, valid for both reads and writes for `len * size_of::<T>()` many bytes, and it must be properly aligned. This means in particular:
  - The entire memory range of this slice must be contained within a single allocation! Slices can never span across multiple allocations.
  - `data` must be non-null and aligned even for zero-length slices or slices of ZSTs. One reason for this is that enum layout optimizations may rely on references (including slices of any length) being aligned and non-null to distinguish them from other data. You can obtain a pointer that is usable as `data` for zero-length slices using `NonNull::dangling()`.
- `data` must point to `len` consecutive properly initialized values of type `T`.
- The memory referenced by the returned slice must not be accessed through any other pointer (not derived from the return value) for the duration of lifetime `'a`. Both read and write accesses are forbidden.
- The total size `len * size_of::<T>()` of the slice must be no larger than `isize::MAX`, and adding that size to `data` must not "wrap around" the address space. See the safety documentation of `pointer::offset`.

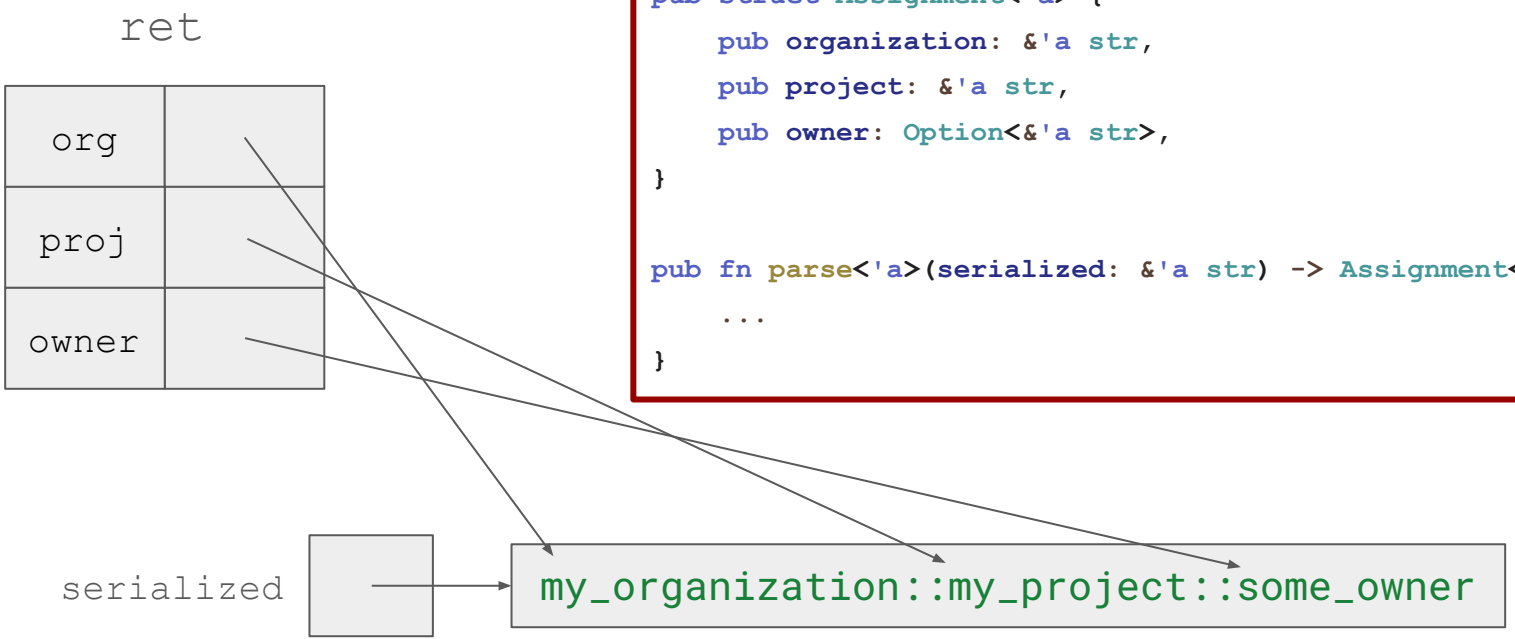
Where Do We Want Unsafe?

Note that `split_at_mut` is a safe abstraction over an unsafe operation.



Another Unsafe Example

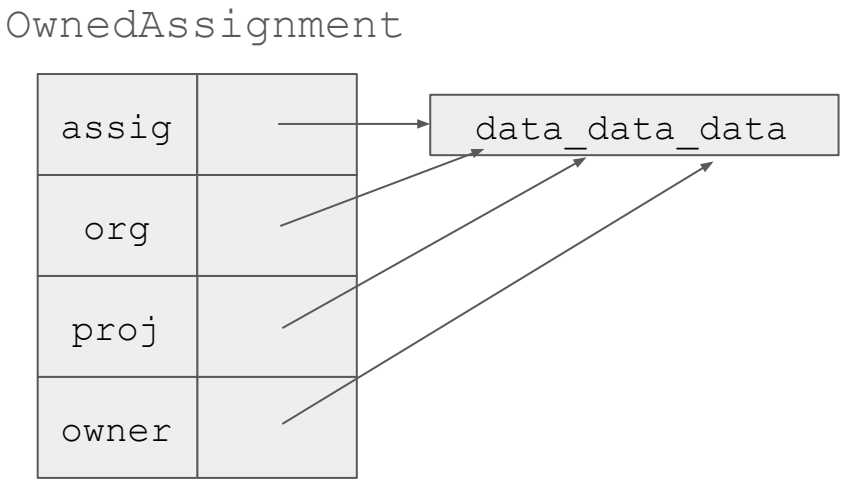
Remember Homework 4, Exercise 5 (the zero-copy parser)?




```rust
pub struct Assignment<'a> {
    pub organization: &'a str,
    pub project: &'a str,
    pub owner: Option<&'a str>,
}

pub fn parse<'a>(serialized: &'a str) -> Assignment<'a> {
    ...
}
```

Another Unsafe Example

Let's make an "owned" version…

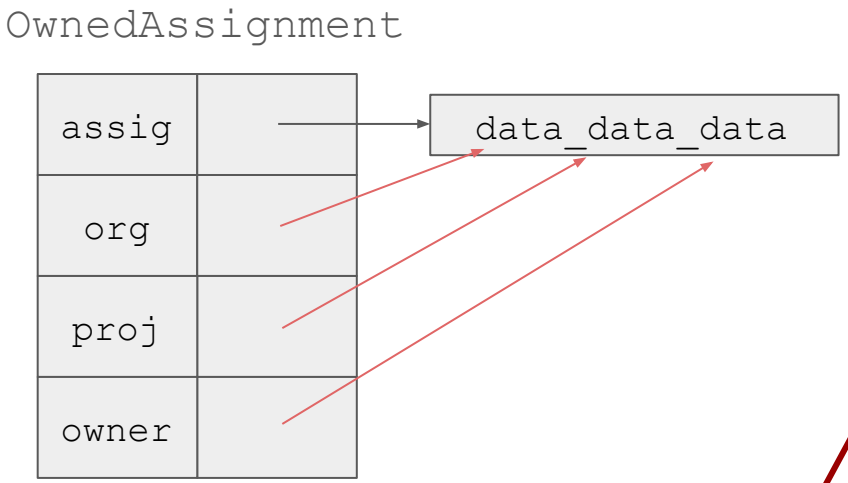



```rust
struct OwnedAssignment<'a> {
    assignment: String,
    org: &'a str,
    proj: &'a str,
    owner: &'a str,
}

impl<'a> OwnedAssignment<'a> {
    pub fn new(assignment: String) -> Self {
        let org   = find_org(&assignment);
        let proj  = find_proj(&assignment);
        let owner = find_owner(&assignment);

        Self { assignment, org, proj, owner }
    }
}
```

This leads to a lot of lifetime errors.

Another Unsafe Example

Pointers allow self-referential structs




```rust
struct OwnedAssignment {
    assignment: String,
    org: *const char,
    proj: *const char,
    owner: *const char,
}

impl OwnedAssignment {
    pub fn new(assignment: String) -> Self {
        let org   = find_org(&assignment);
        let proj  = find_proj(&assignment);
        let owner = find_owner(&assignment);

        Self { assignment, org, proj, owner }
    }
}
```

Can now define a safe interface here!

Another Unsafe Example

For async/await, the compiler-generated futures are *sometimes* self-referential!

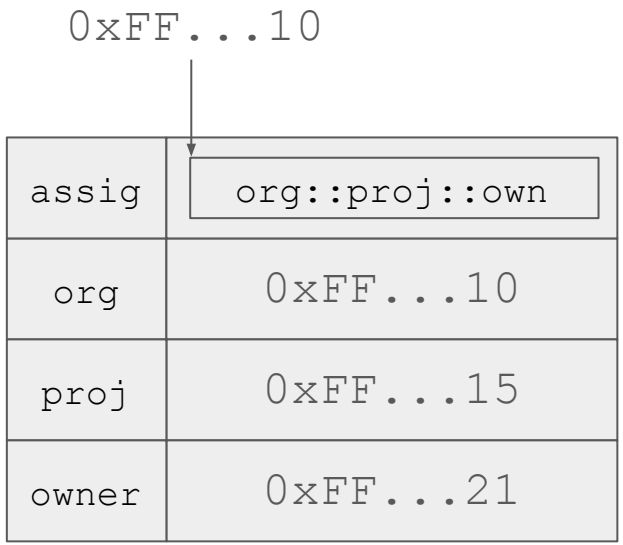
```rust
async fn send(name: &str) { ... }

async fn send_name(full_name: String) {
    let first_name = full_name
        .split(" ")
        .next()
        .unwrap();

    send(first_name).await;

    println!("Sent {first_name}!");
}
```

```rust
enum SendNameState {
    Start {
        full_name: String,
    },
    Await1 {
        full_name: String,
        first_name: &str,
    },
    Done,
}
```

Note: this is not a perfect representation of what goes on “underneath the hood”

Why Bring Up Parse, Don’t Validate?

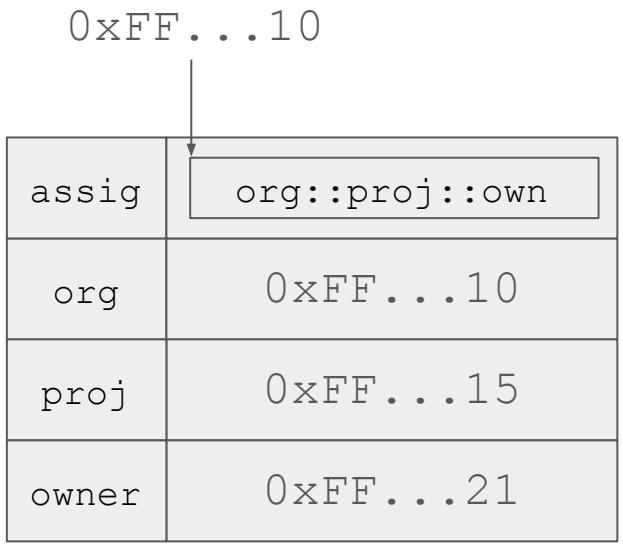
Self referential structs have another problem beyond reference lifetimes…




```
fn main() {
    let a1 = OwnedAssignment::parse("assig::org::own");
    ...
    let a2 = a1;
    ...
}
```

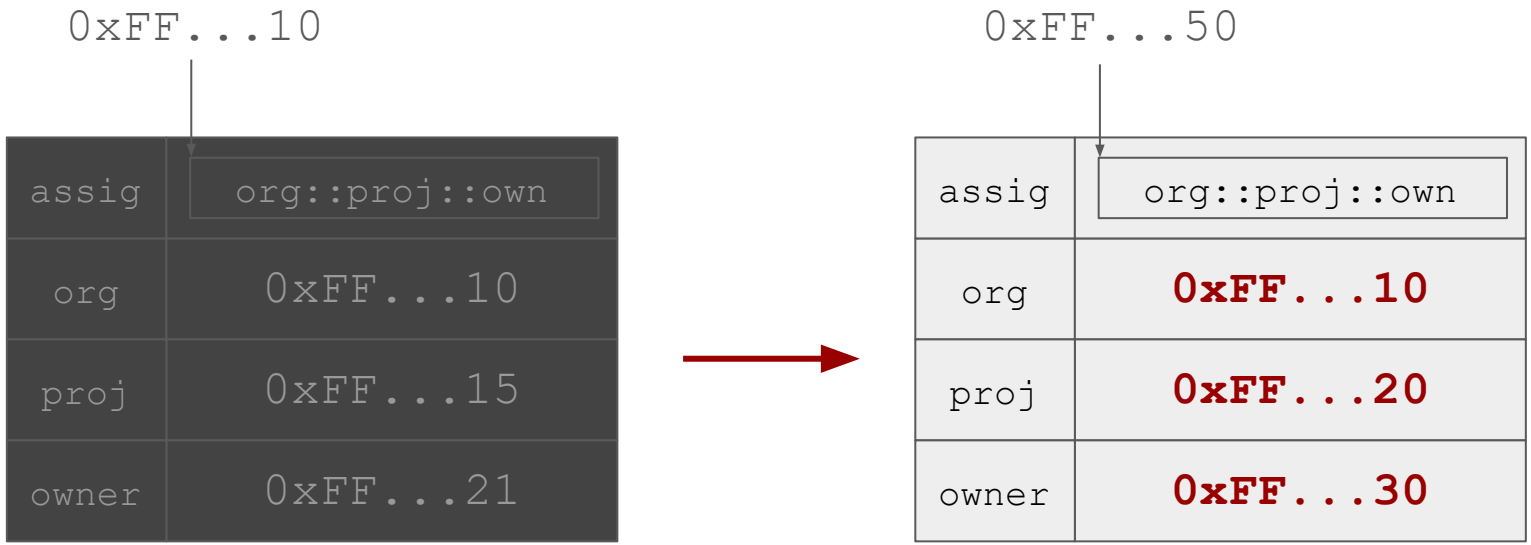
Why Bring Up Parse, Don’t Validate?

Self referential structs have another problem beyond reference lifetimes…



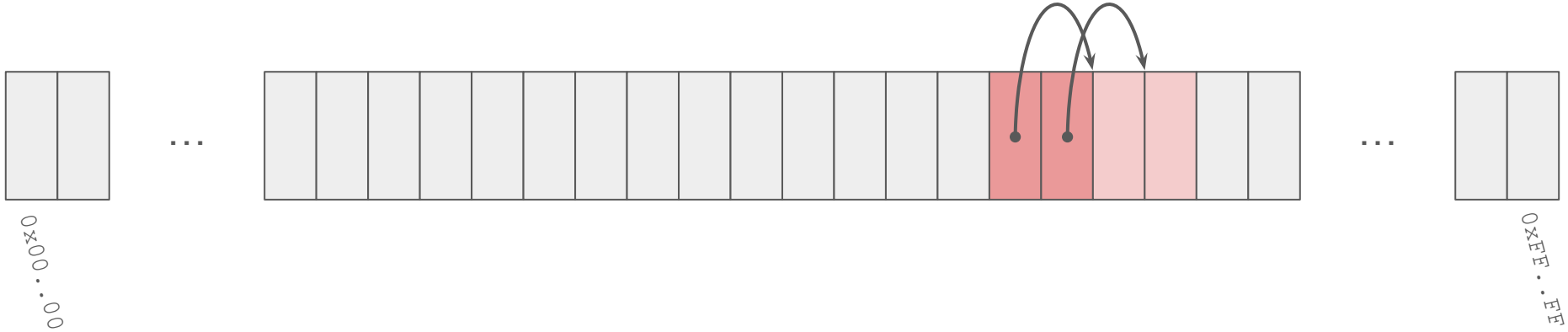
Why Bring Up Parse, Don’t Validate?

Self referential structs have another problem beyond reference lifetimes…



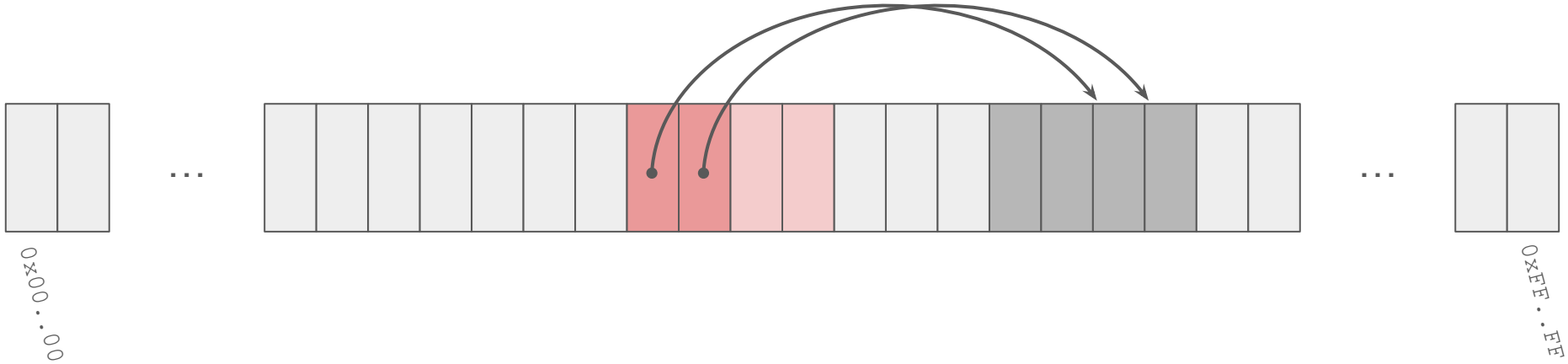
Why Bring Up Parse, Don’t Validate?

Self referential structs have another problem beyond reference lifetimes…



Why Bring Up Parse, Don’t Validate?

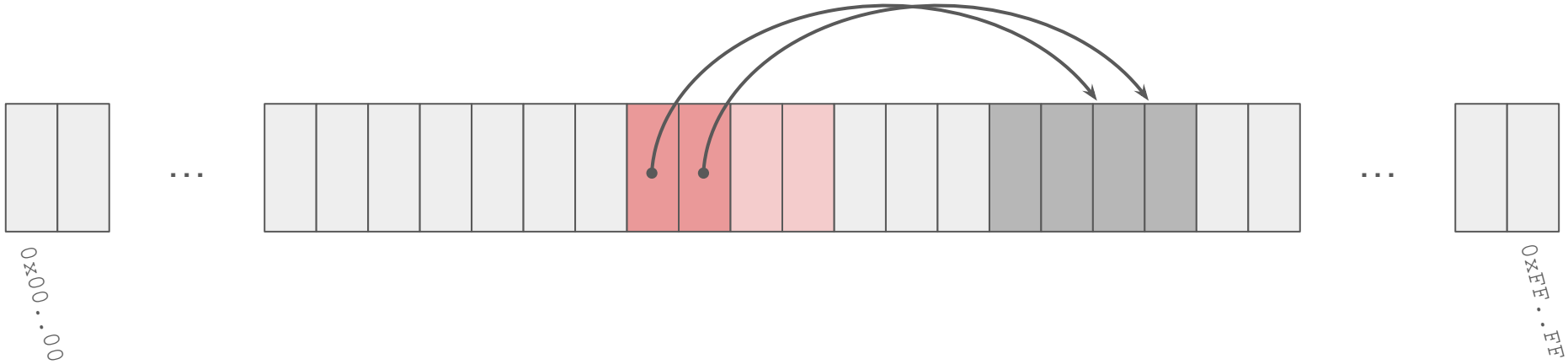
Self referential structs have another problem beyond reference lifetimes…



So What Can We Do?

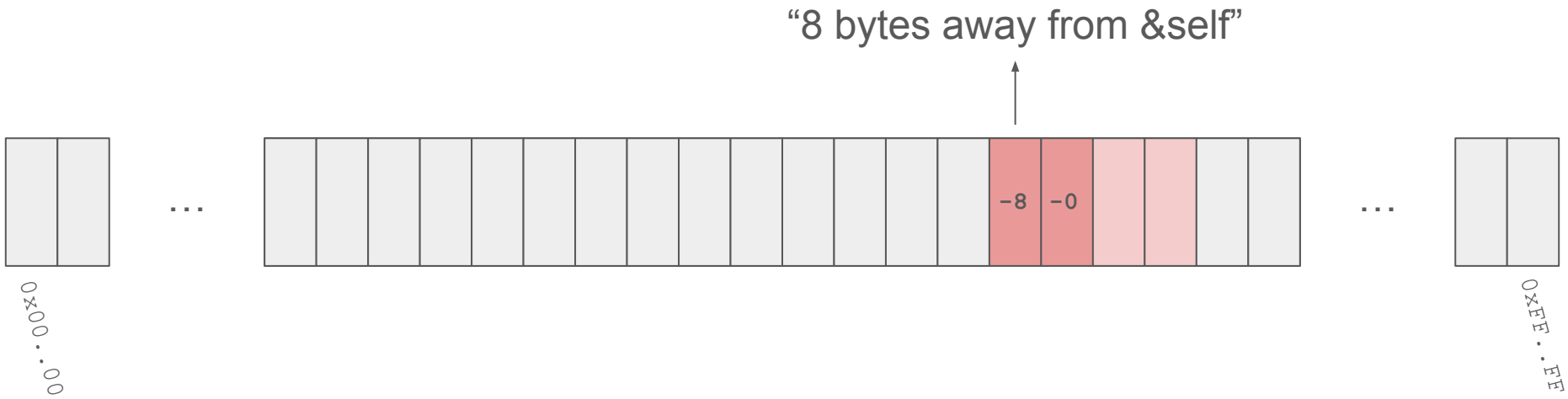
Rust ran into this exact problem specifically with `async`/`await` in 2018…

- The compiler assumes it can freely use moves to optimize as necessary
- Yet `Future`s cannot be moved…



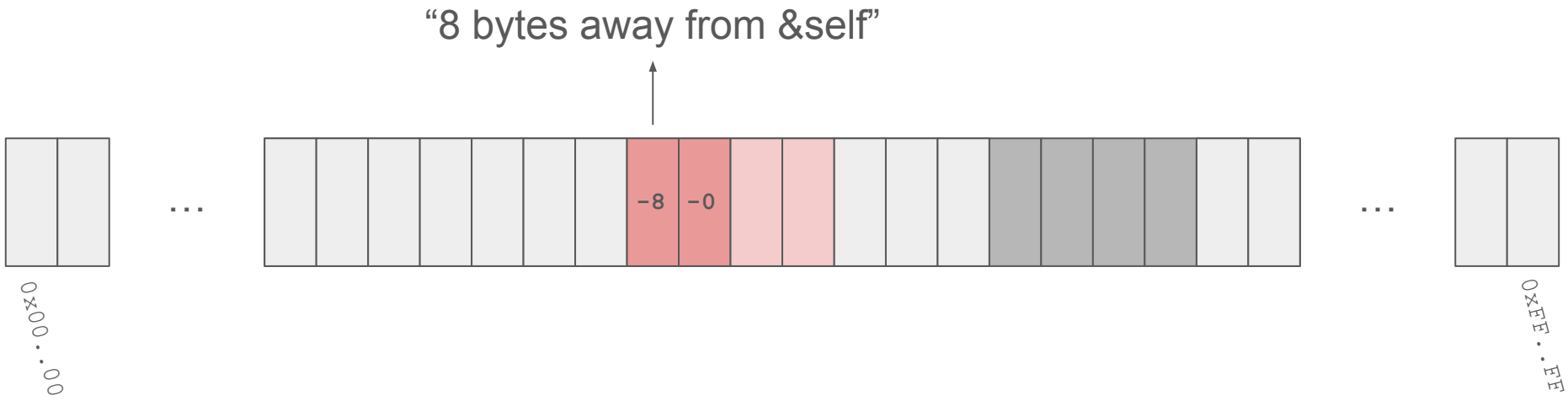
So What Can We Do?

Make structs use “relative pointers” → store offsets, not addresses



So What Can We Do?

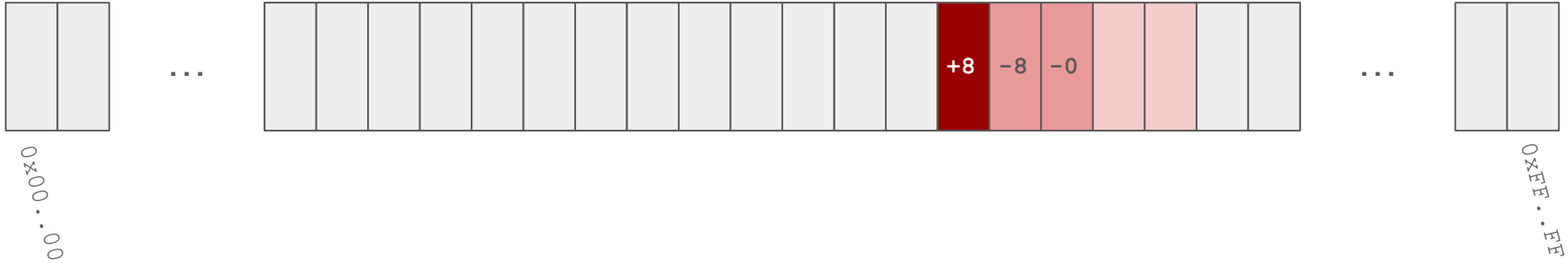
Make structs use “relative pointers” → store offsets, not addresses



So What Can We Do?

Make structs use “relative pointers” → store offsets, not addresses

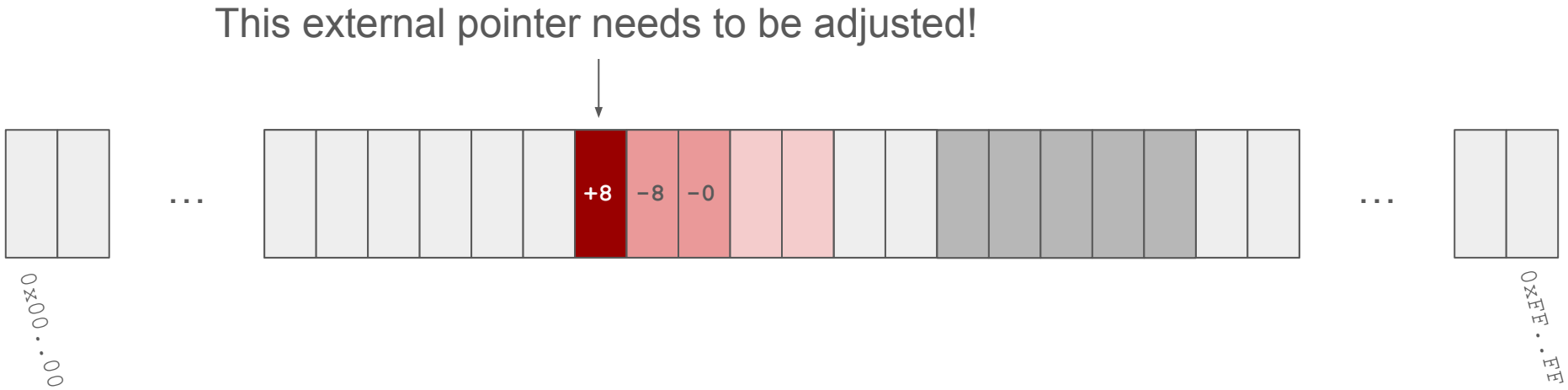
- Some pointers still require fixing



So What Can We Do?

Make structs use “relative pointers” → store offsets, not addresses

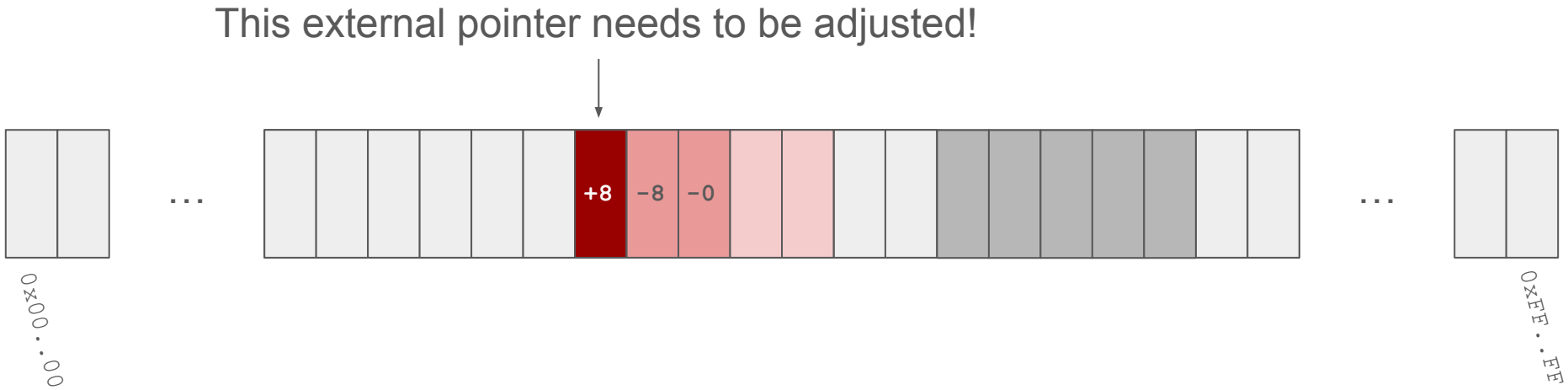
- Some pointers still require fixing



So What Can We Do?

Make structs use “relative pointers” → store offsets, not addresses

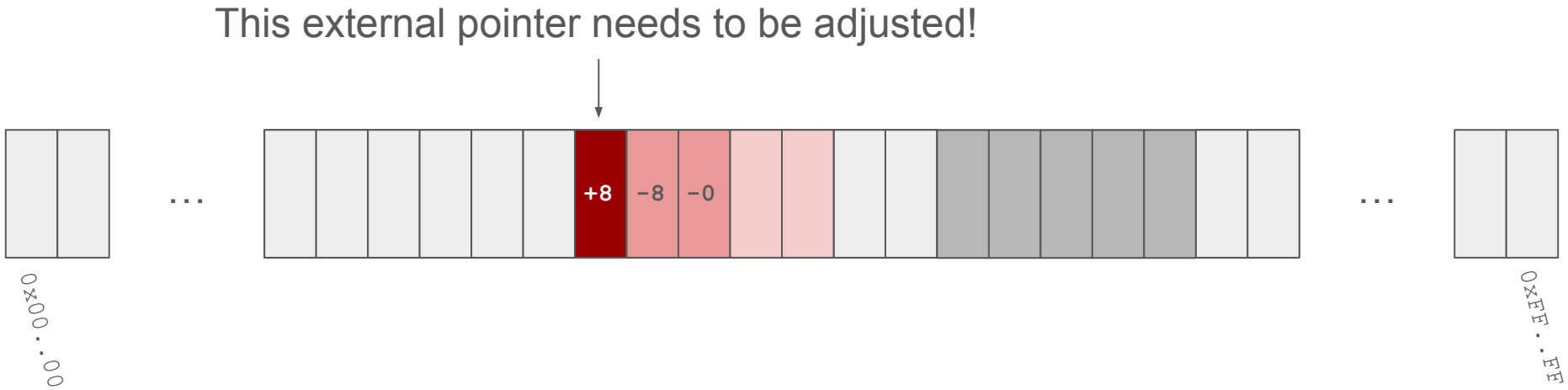
- Some pointers still require fixing… and we perform moves really often



So What Can We Do?

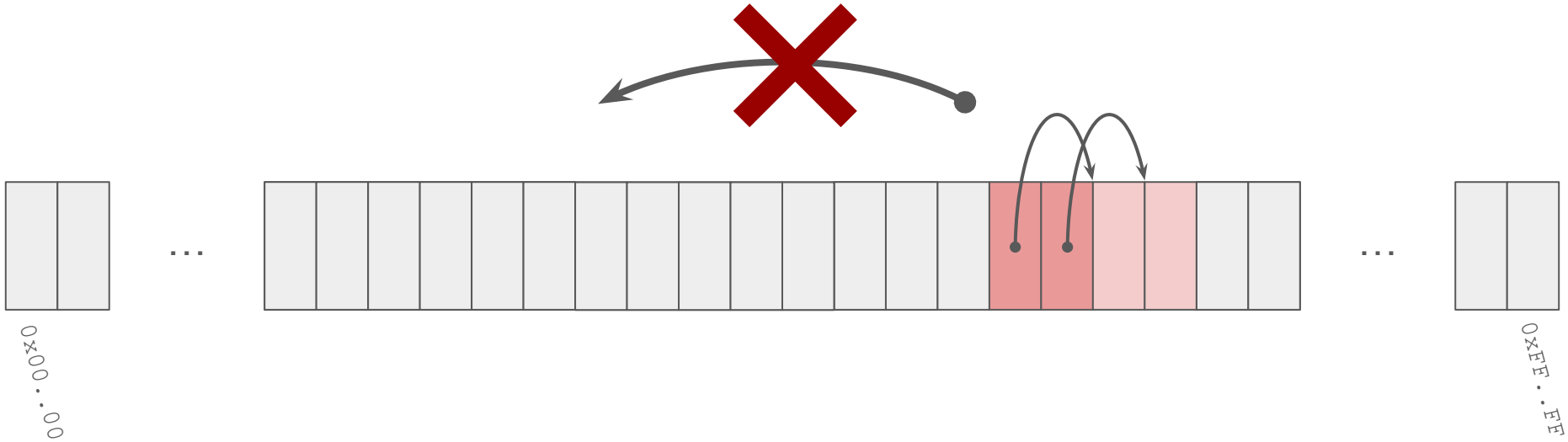
Make structs use “relative pointers” → store offsets, not addresses

- Some pointers still require fixing… and we perform moves really often
- Non-standard memory layout results in ABI/FFI issues



So What Can We Do?

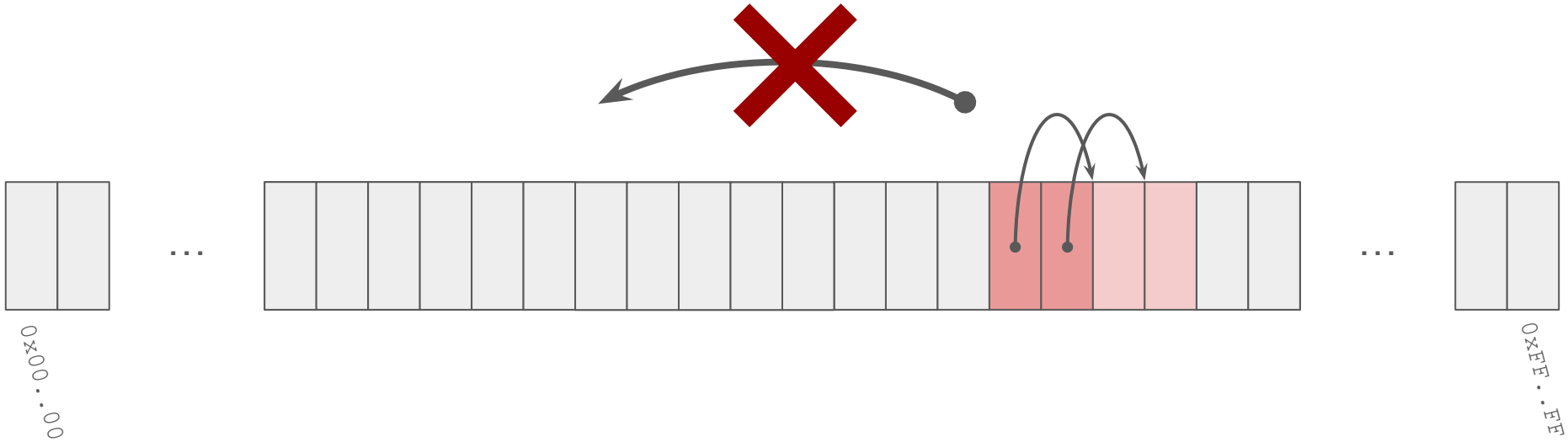
…… what if we just don’t move self-referential structs around?



So What Can We Do?

…… what if we just don’t move self-referential structs around?

- Create a new mechanism to disallow it!
  - An invariant enforced by the type system



Introducing `Pin<Ptr>`

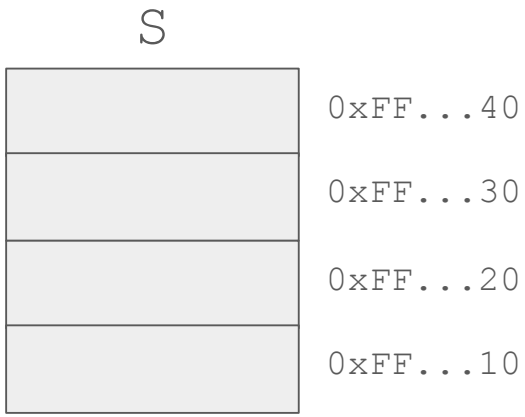
Let’s look at a common life-cycle for self-referential types

1. Memory is allocated to accommodate struct `S`

Introducing `Pin<Ptr>`

Let’s look at a common life-cycle for self-referential types

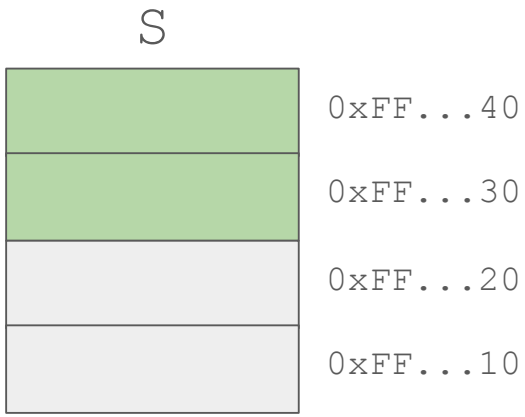
1. Memory is allocated to accommodate struct `S`



Introducing `Pin<Ptr>`

Let's look at a common life-cycle for self-referential types

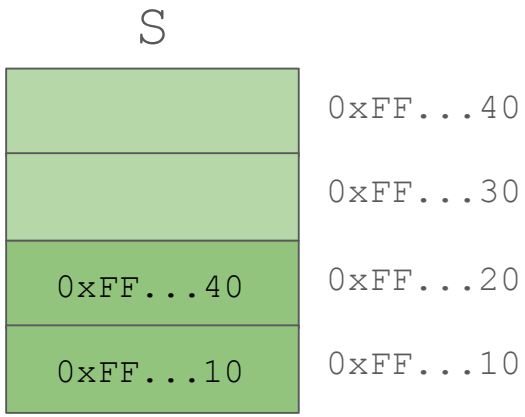
1. Memory is allocated to accommodate struct `S`
2. `S`'s data fields are initialized



Introducing `Pin<Ptr>`

Let's look at a common life-cycle for self-referential types

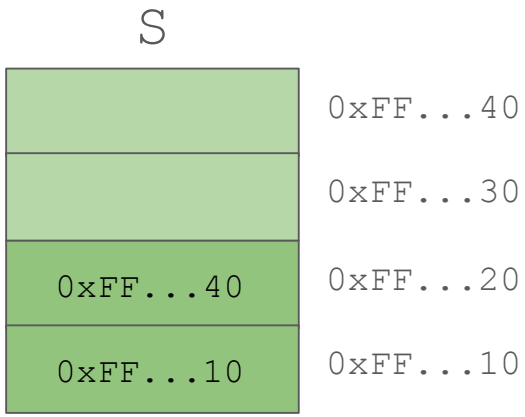
1. Memory is allocated to accommodate struct `S`
2. `S`'s data fields are initialized
3. `S`'s self-referential pointer fields are initialized



Introducing `Pin<Ptr>`

Let's look at a common life-cycle for self-referential types

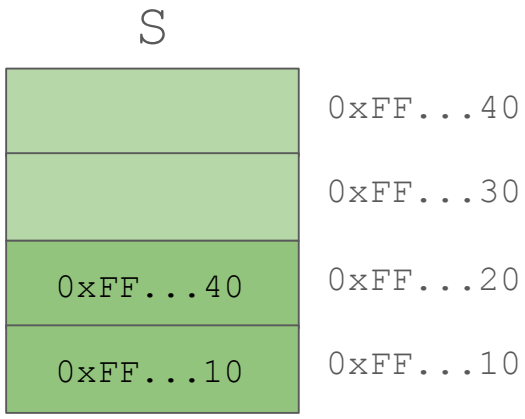
1. Memory is allocated to accommodate struct `S`
2. `S`'s data fields are initialized
3. `S`'s self-referential pointer fields are initialized
4. `S` is used to do stuff



Introducing `Pin<Ptr>`

Let's look at a common life-cycle for self-referential types

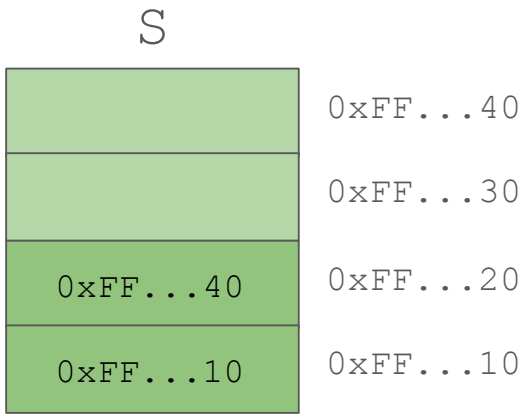
1. Memory is allocated to accommodate struct `S`
2. `S`'s data fields are initialized
3. `S`'s self-referential pointer fields are initialized
4. `S` is used to do stuff
5. `S` is cleaned up



Introducing `Pin<Ptr>`

Let's look at a common life-cycle for self-referential types

1. Memory is allocated to accommodate struct `S`
2. `S`'s data fields are initialized
3. `S`'s self-referential pointer fields are initialized
4. `S` is used to do stuff
5. `S` is cleaned up

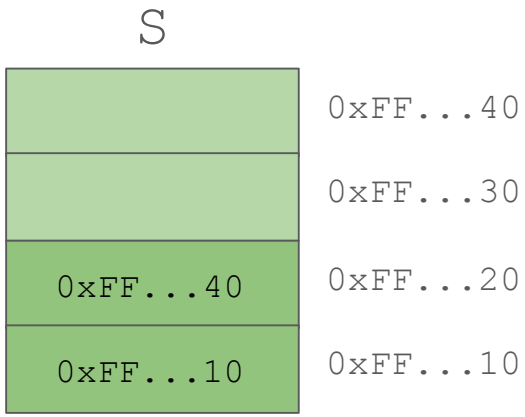


When can `S` be moved? When can it not?

Introducing `Pin<Ptr>`

Let's look at a common life-cycle for self-referential types

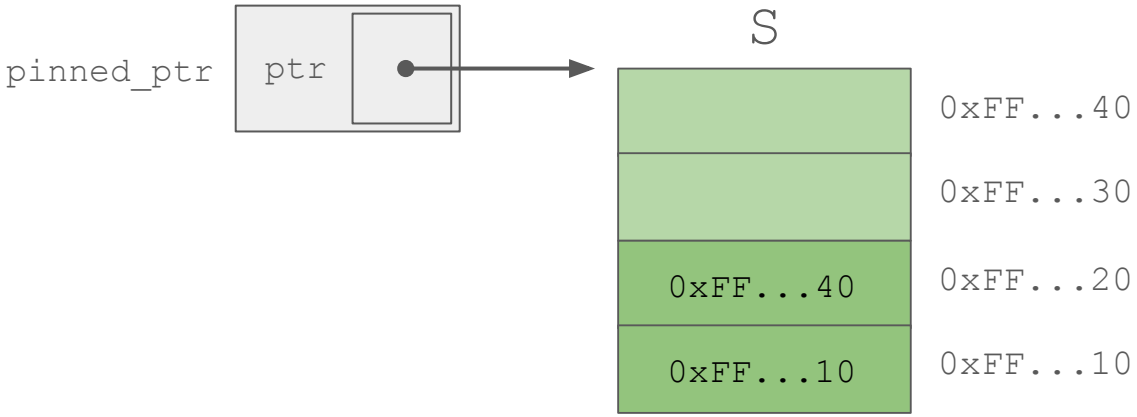
1. Memory is allocated to accommodate struct `S`
2. `S`'s data fields are initialized
3. `S`'s self-referential pointer fields are initialized
4. `S` is used to do stuff
5. `S` is cleaned up



When can `S` be moved? When can it not?

Introducing `Pin<Ptr>`

Once `S` can no longer be moved, let's create a pointer to `S`, and wrap it in `Pin`

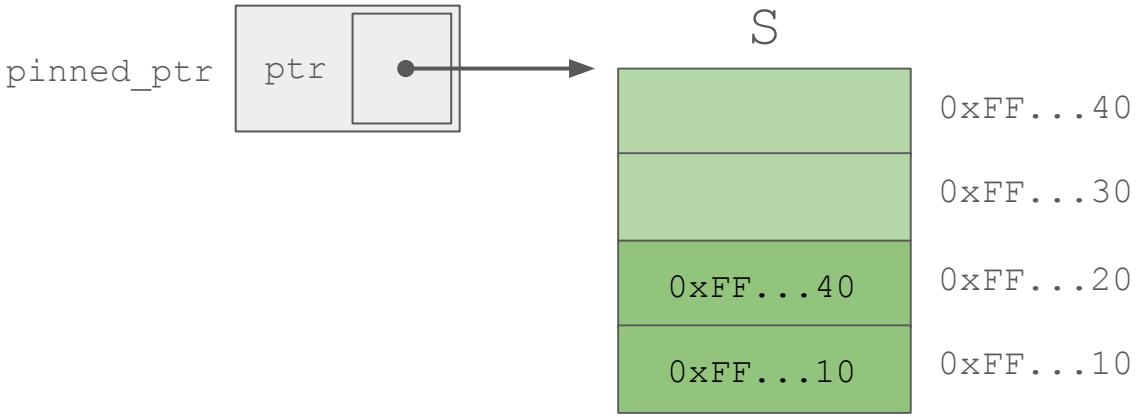


`Pin<Ptr>` enforces that for the `Pin`'s lifetime, `Ptr`'s `Target` won't be moved!

Introducing `Pin<Ptr>`

* Doesn't just need to be `Box`, can be any "pointer type" including `&`, `&mut`, `Rc`, `Arc`, etc…

Once `S` can no longer be moved, let's create a pointer to `S`, and wrap it in `Pin`

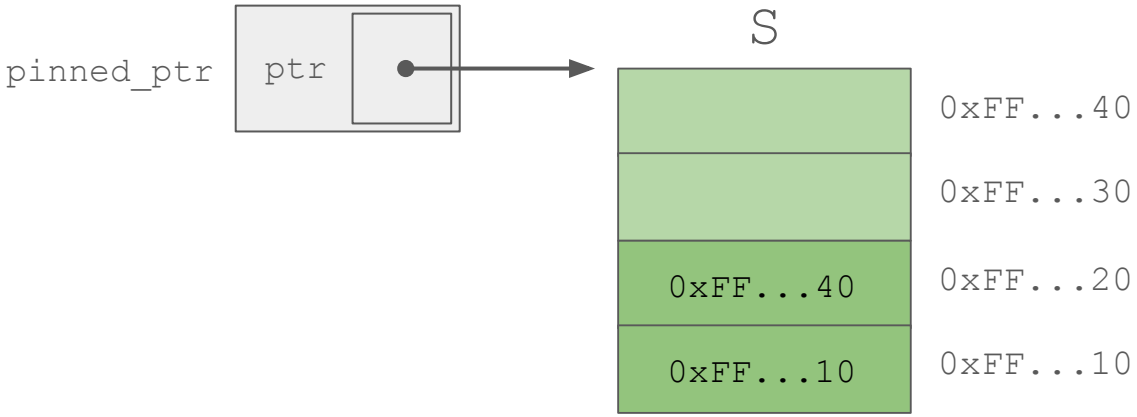


`Pin<Ptr>` enforces that for the `Pin`'s lifetime, `Ptr`'s `Target` won't be moved!

- All functions that interact with `S` *and need to rely on `S` not moving*, will access it through `Pin<Box<S>>`*

Introducing `Pin<Ptr>`

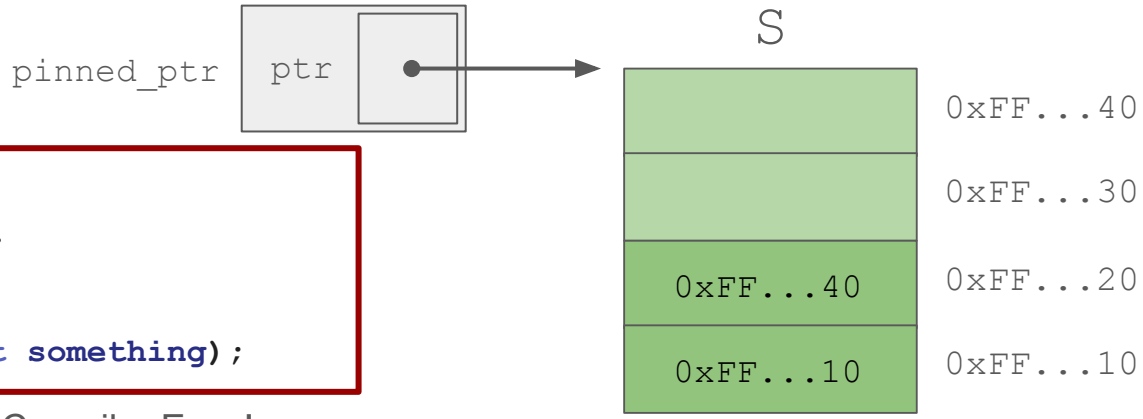
Once `S` can no longer be moved, let's create a pointer to `S`, and wrap it in `Pin`



Now that we've pinned `S`, how do we guarantee it won't be moved?

Introducing `Pin<Ptr>`

Once `S` can no longer be moved, let's create a pointer to `S`, and wrap it in `Pin`




```
let pin = Box::pin(S{ ... });
let mut something = S { ... };

std::mem::swap(&mut *pin, &mut something);
```

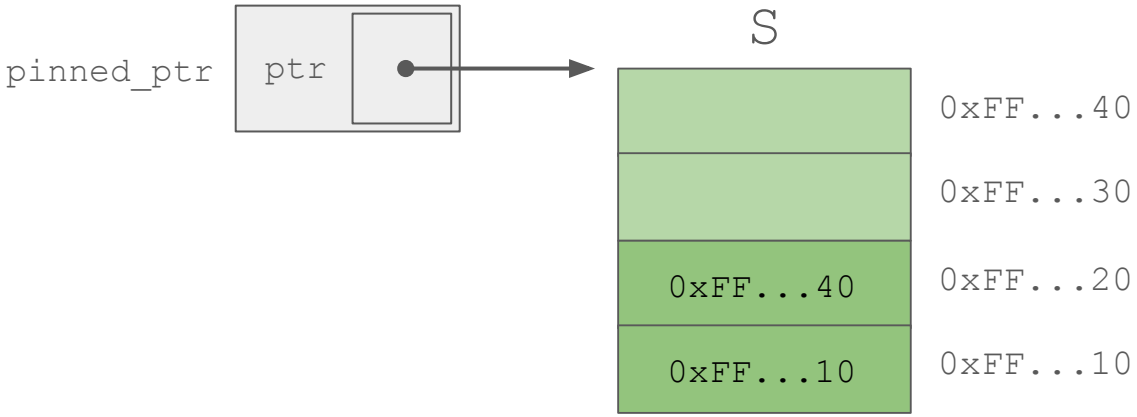
Compiler Error!

Now that we've pinned `S`, how do we guarantee it won't be moved?

- Moving `S` would require getting a mutable reference to it, so let's explicitly NOT implement `DerefMut` on `Pin<Ptr>`

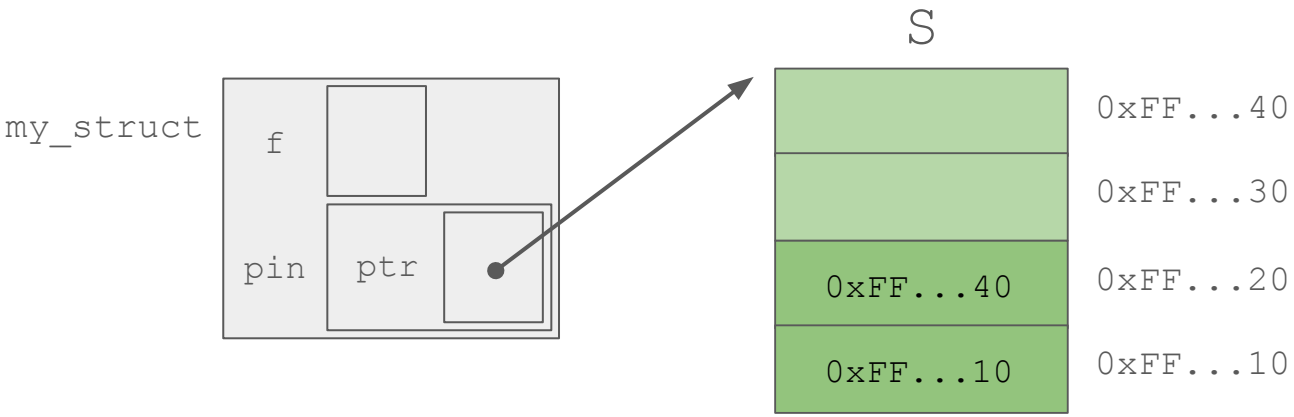
Introducing `Pin<Ptr>`

Why pin a pointer? Why not “pin” the underlying `Target`?



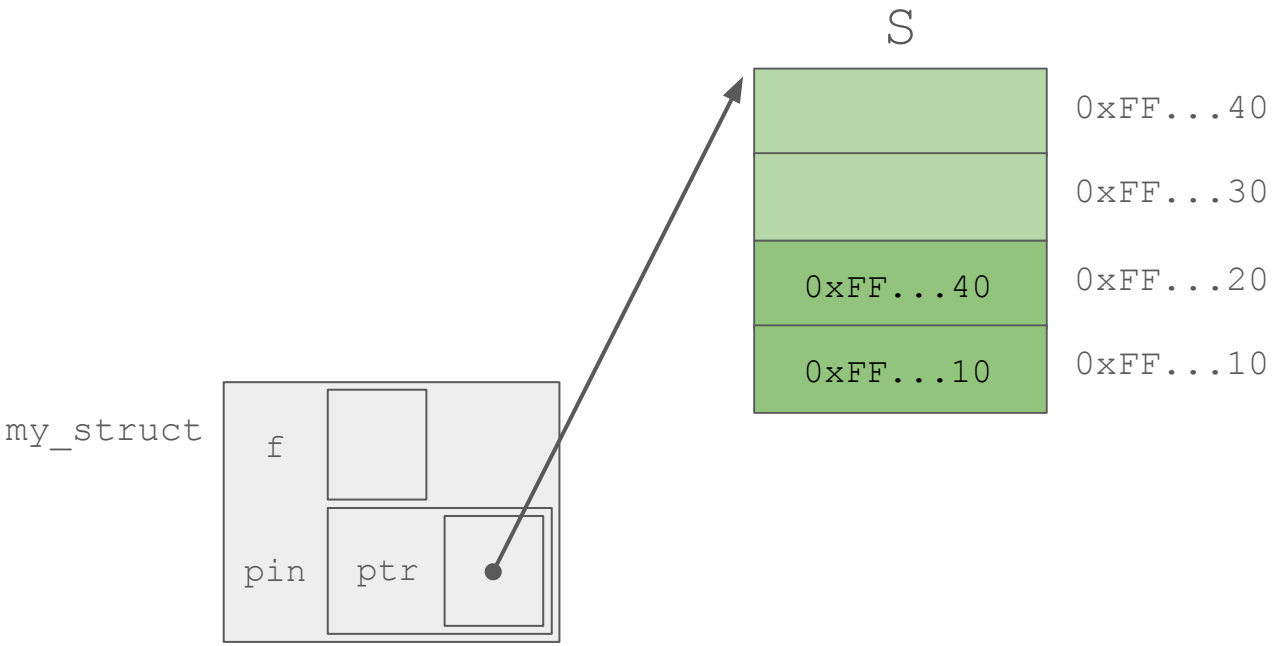
Introducing `Pin<Ptr>`

Why pin a pointer? Why not “pin” the underlying `Target`?



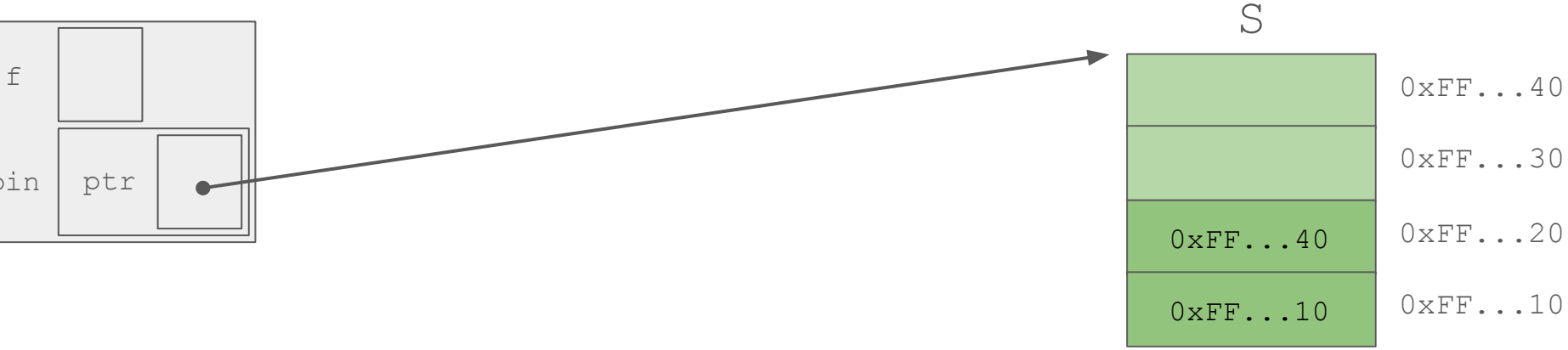
Introducing `Pin<Ptr>`

Why pin a pointer? Why not “pin” the underlying `Target`?



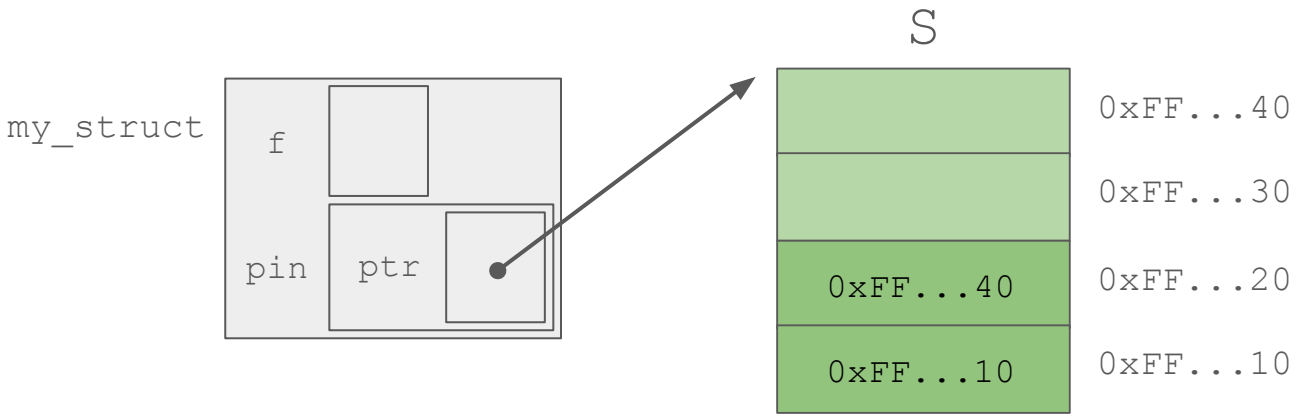
Introducing `Pin<Ptr>`

Why pin a pointer? Why not “pin” the underlying `Target`?



Introducing `Pin<Ptr>`

Why pin a pointer? Why not “pin” the underlying `Target`?



If `S` itself was pinned, then we wouldn’t be able to move around `my_struct`!

Constructing `Pin<Ptr>`

Let’s take a look at how you actually make one of these…

```
impl<Ptr> Pin<Ptr>                                                    Source
where
    Ptr: Deref,

pub const unsafe fn new_unchecked(pointer: Ptr) -> Pin<Ptr>   1.33.0 (const: 1.84.0) · Source
```

Constructs a new `Pin<Ptr>` around a reference to some data of a type that may or may not implement `Unpin`.

If `pointer` dereferences to an `Unpin` type, `Pin::new` should be used instead.

Safety

This constructor is unsafe because we cannot guarantee that the data pointed to by `pointer` is pinned. At its core, pinning a value means making the guarantee that the value’s data will not be moved nor have its storage invalidated until it gets dropped. For a more thorough explanation of pinning, see the `pin` module docs.

If the caller that is constructing this `Pin<Ptr>` does not ensure that the data `Ptr` points to is pinned, that is a violation of the API contract and may lead to undefined behavior in later (even safe) operations.

Constructing `Pin<Ptr>`

Let’s take a look at how you actually make one of these…

```
pub fn main() {
    let s = S { ... };
    let box_s = Box::new(s);
    let pinned_s = unsafe { Pin::new_unchecked(box_s) };
}
```

Constructing `Pin<Ptr>`

Let's take a look at how you actually make one of these…

```
pub fn main() {
    let s = S { ... };
    let box_s = Box::new(s);
    let pinned_s = unsafe { Pin::new_unchecked(box_s) };
}
```

Why is this an unsafe function?

Constructing `Pin<Ptr>`

Let's take a look at how you actually make one of these…

```
pub fn main() {
    let mut s = S { ... };
    let mut something = S { ... };

    let pinned_s = unsafe { Pin::new_unchecked(&mut s) };
    let s_ref = &mut s;

    std::mem::swap(s_ref, &mut something)
}
```

Constructing `Pin<Ptr>`

Let's take a look at how you actually make one of these…

```
pub fn main() {
    let mut s = S { ... };
    let mut something = S { ... };

    let pinned_s = unsafe { Pin::new_unchecked(&mut s) };
    let s_ref = &mut s;

    std::mem::swap(s_ref, &mut something)
}
```

We made an unsafe promise and then broke the pin invariant!

Constructing `Pin<Ptr>`

Let's take a look at how you actually make one of these…

```
pub fn main() {
    let s = S { ... };
    let box_s = Box::new(s);
    let pinned_s = unsafe { Pin::new_unchecked(box_s) };
}
```

`Box` has a very similar operation already defined on it!

```
impl<T> Box<T> {
    pub fn pin(x: T) -> Pin<Box<T>> {
        Box::new(x).into()
    }
}
```

Constructing `Pin<Ptr>`

Let's take a look at how you actually make one of these…

```
pub fn main() {
    let s = S { ... };
    let box_s = Box::new(s);
    let pinned_s = unsafe { Pin::new_unchecked(box_s) };
}
```

`Box` has a very similar operation already defined on it!

Why is `Box::pin()` a safe function?

```
impl<T> Box<T> {
    pub fn pin(x: T) -> Pin<Box<T>> {
        Box::new(x).into()
    }
}
```

Constructing `Pin<Ptr>`

Let's take a look at how you actually make one of these…

```
pub fn main() {
    let s = S { ... };
    let box_s = Box::new(s);
    let pinned_s = unsafe { Pin::new_unchecked(box_s) };
}
```

`Box` has a very similar operation already defined on it!

```
impl<T> Box<T> {
    pub fn pin(x: T) -> Pin<Box<T>> {
        Box::new(x).into()
    }
}
```

Why is `Box::pin()` a safe function?

- Ownership of `x` is taken by `pin`

Coming Back to Async/Await

Things that can be `await`-ed implement the `Future` trait

```
pub trait Future {
    #[lang = "poll"]
    #[stable(feature = "futures_api", since = "1.36.0")]
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}
```

Coming Back to Async/Await

Things that can be `await`-ed implement the `Future` trait

```
pub trait Future {
    #[lang = "poll"]
    #[stable(feature = "futures_api", since = "1.36.0")]
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}
```

You can construct `MyFuture` however you want…

- But you *cannot* call `poll()` without pinning it first!
- `poll()` can work knowing that the underlying `MyFuture` will not move!

Coming Back to Async/A

Things that can be `await`-ed impl

```
pub trait Future {
    #[lang = "poll"]
    #[stable(feature = "futures_ap
    fn poll(self: Pin<&mut Self>, c
}
```

You can construct `MyFuture` howev

- But you *cannot* call `poll()` with
- `poll()` can work knowing that th

Introducing `Pin<Ptr>`

Let's look at a common life-cycle for self-referent

1. Memory is allocated to accommodate stru
2. `S`'s data fields are initialized
3. `S`'s self-referential pointer fields are initiali
4. `S` is used to do stuff
5. `S` is cleaned up

When can `S` be moved? When can it not?

Making the Invariant Better

Let's give S some specific data, and consider the following function

```
struct S {
    a: u32,
    b: f32,
}
```

```
fn foo(my_pin: &Pin<Box<S>>) { ... }

pub fn main() {
    let my_struct = S {
        a: 10,
        b: 20.0,
    };

    // I want to call foo!
}
```

`foo()` needs to operate assuming that `S` will not be moved

Making the Invariant Better

Let's give S some specific data, and consider the following function

```rust
struct S {
    a: u32,
    b: f32,
}
```

```rust
fn foo(my_pin: &Pin<Box<S>>) { ... }

pub fn main() {
    let my_struct = S {
        a: 10,
        b: 20.0,
    };
    let mut pin = Box::pin(my_struct);
    foo(&pin);
}
```

Making the Invariant Better

Let's give S some specific data, and consider the following function

```
struct S {
    a: u32,
    b: f32,
}
```

```
fn foo(my_pin: &Pin<Box<S>>) { ... }

pub fn main() {
    let my_struct = S {
        a: 10,
        b: 20.0,
    };
    let mut pin = Box::pin(my_struct);
    foo(&pin);
}
```

Is there anything actually wrong with moving `S`?

Making the Invariant Better

Let's give S some specific data, and consider the following function

```
struct S {
    a: u32,
    b: f32,
}
```

```
pub fn main() {
    let my_struct = S {
        a: 10,
        b: 20.0,
    };
    let mut pin = Box::pin(my_str
    foo(&pin);
    ...
    let boxed_struct = Pin::into_inner(pin);
}
```

*S is not actually address-sensitive, I **should** be able to do this!*

Making the Invariant Better

Let's give S some specific data, and consider the following function

```
struct S {
    a: u32,
    b: f32,
}
```

```
pub fn main() {
    let my_struct = S {
        a: 10,
        b: 20.0,
    };
    let mut pin = Box::pin(my_str
    foo(&pin);
    ...
    let boxed_struct = Pin::into_inner(pin);
}
```

*Pin is giving **too strong** of a guarantee!*
Even movable types are being restricted…

S is not actually address-sensitive,
*I **should** be able to do this!*

Unpin

Rust decided to pair `Pin<Ptr>` with a marker trait `Unpin`

std::marker

Trait Unpin

Since 1.33.0 · Source

Search Settings Help Summary

```
pub auto trait Unpin { }
```

Types that do not require any pinning guarantees.

For information on what "pinning" is, see the `pin` module documentation.

Implementing the `Unpin` trait for `T` expresses the fact that `T` is pinning-agnostic: it shall not expose nor rely on any pinning guarantees. This, in turn, means that a `Pin`-wrapped pointer to such a type can feature a *fully unrestricted* API. In other words, if `T: Unpin`, a value of type `T` will *not* be bound by the invariants which pinning otherwise offers, even when "pinned" by a `Pin<Ptr>` pointing at it. When a value of type `T` is pointed at by a `Pin<Ptr>`, `Pin` will not restrict access to the pointee value like it normally would, thus allowing the user to do anything that they normally could with a non-`Pin`-wrapped `Ptr` to that value.

Unpin

Rust decided to pair `Pin<Ptr>` with a marker trait `Unpin`

```
struct S {
    a: u32,
    b: f32,
}
```

`u32` and `f32` both implement `Unpin`, therefore `S` implements `Unpin`

```
pub fn main() {
    let my_struct = S {
        a: 10,
        b: 20.0,

    n = Box::pin(my_struct);
    foo(&pin);
    ...
    let boxed_struct = Pin::into_inner(pin);
}
```

Unpin

Rust decided to pair `Pin<Ptr>` with a marker trait `Unpin`

```
struct S {
    a: u32,
    b: f32,
}
```

```
pub fn main() {
    let my_struct = S {
        a: 10,
        b: 20.0,
    }

    n = Box::pin(my_struc
    foo(&pin);
    ...
    let boxed_struct = Pin::into_inner(pin);
}
```

`u32` and `f32` both implement `Unpin`, therefore `S` implements `Unpin`

Which makes this line compile!

Unpin

To make a struct not implement `Unpin`, include the `PhantomPinned` Zero-Sized Type!

```
struct S {
    a: u32,
    b: f32,
}
```

```
struct S {
    field: u32,
    field_ptr: *const u32,
    // This makes the struct !Unpin
    _pin: PhantomPinned,
}
```

— or —

```
impl !Unpin for S {}
```

Pinning Unpin Types

A `Pin<Ptr>` where `Ptr` points to a `T: Unpin` allows the `Pin` to be ignored

Implementations

```
impl<Ptr> Pin<Ptr>
where
    Ptr: Deref,
    <Ptr as Deref>::Target: Unpin,
```

```
pub const fn new(pointer: Ptr) -> Pin<Ptr>
```
1.33.0 (const: 1.84.0) · Source

```
pub const fn into_inner(pin: Pin<Ptr>) -> Ptr
```
1.39.0 (const: 1.84.0) · Source

I can safely construct a `Pin` if the pinned `Target` is `Unpin`

```
impl<Ptr> Pin<Ptr>
where
    Ptr: Deref,
```
Source

```
pub const unsafe fn new_unchecked(pointer: Ptr) -> Pin<Ptr>
```
1.33.0 (const: 1.84.0) · Source

```
pub fn as_ref(&self) -> Pin<&<Ptr as Deref>::Target>
where
    Ptr: Deref,
```
1.33.0 (const: unstable) · Source

Pinning Unpin Types

A `Pin<Ptr>` where `Ptr` points to a `T: Unpin` allows the `Pin` to be ignored.

Implementations

```
impl<Ptr> Pin<Ptr>
where
    Ptr: Deref,
    <Ptr as Deref>::Target: Unpin,

pub const fn new(pointer: Ptr) -> Pin<Ptr>                    1.33.0 (const: 1.84.0) · Source

pub const fn into_inner(pin: Pin<Ptr>) -> Ptr                 1.39.0 (const: 1.84.0) · Source

impl<Ptr> Pin<Ptr>                                            Source
where
    Ptr: Deref,

pub const unsafe fn new_unchecked(pointer: Ptr) -> Pin<Ptr>   1.33.0 (const: 1.84.0) · Source

pub fn as_ref(&self) -> Pin<&<Ptr as Deref>::Target>          1.33.0 (const: unstable) · Source
where
    Ptr: Deref,
```

I can safely extract the underlying pointer, if the pointed-to `Target` is `Unpin`

Pinning Unpin Types

A `Pin<Ptr>` where `Ptr` points to a `T: Unpin` allows the `Pin` to be ignored.

Implementations

```
impl<Ptr> Pin<Ptr>                                                    Source
where
    Ptr: Deref,
    <Ptr as Deref>::Target: Unpin,

  pub const fn new(pointer: Ptr) -> Pin<Ptr>
  pub const fn into_inner(pin: Pin<Ptr>) -> Ptr

impl<Ptr> Pin<Ptr>
where
    Ptr: Deref,

  pub const unsafe fn new_unchecked(pointer: Ptr) -> Pin<Ptr>          1.33.0 (const: 1.84.0) · Source
  pub fn as_ref(&self) -> Pin<&<Ptr as Deref>::Target>                 1.33.0 (const: unstable) · Source
  where
      Ptr: Deref,
```

*I can **always** construct a `Pin<Ptr>` with `new_unchecked` even if `Target` is not `Unpin`*

Comparing the Two Solutions

Let’s directly compare relative pointers versus pinned pointers as a solution

Relative Pointers	Pinned Pointers
- Provides safety guarantees	- Provides safety guarantees
- Easy-to-use	- Harder-to-use
- Incurs runtime costs for moves	- Zero-cost abstraction, no runtime cost
- Incurs runtime cost when moving “Unpin” types	- No performance penalty for types which are Unpin
- Non-standard memory layout	- Uses a standard memory layout

Coming Back to Parse, Don't Validate

We looked at two examples of type-specified invariants:

`NonEmptyVec<T>` → *Data* Invariant

A `NonEmptyVec<T>` *contains at least one* `T`

`Pin<Ptr>` → *Behavior* Invariant

If `Ptr` *points to an address-sensitive type* `T`*, then* `T` *will not be moved*

Good type-driven code takes advantage of both kinds of invariants

Takeaways, Further Topics

not feedback, rust is kind of annoying to code.

Each line of code interacts with more than just data

- You have to consider both safety and correctness invariants

Takeaways, Further Topics

not feedback, rust is kind of annoying to code.

Each line of code interacts with more than just data

- You have to consider both safety and correctness invariants

I think the Rust lectures in class are a little scatterbrained and hard to follow at times. I would like to see more cohesion and justification for the various choices Rust makes. It is easy enough to look up something in the Rust book - class should focus on the *why*, as well as the *how* of how to write idiomatic Rust.

Why: each line of code is a “proof of safety”, and benefits from assumptions

How: the type system is robust, and can enforce data and behavior invariants

Takeaways, Further Topics

Writing idiomatic Rust is different that writing code in any other language

- Parse, Don’t Validate: invariants belong in your types.
  - Impossible states should be made impossible to represent.
  - Impossible behaviors should be made impossible to perform.

For the curious few, some more related topics:

- Structural Pinning
  - Pin Projection
- Complex trait bounds with ! and ? operators
- New Type Pattern (`Address` in HW7)
- We barely touched unsafe, there’s a lot more to talk about there…

References

Parse, Don’t Validate
https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/

Reddit: Good Invariants To Keep In Mind
https://www.reddit.com/r/rust/comments/1f1vlr3/good_invariants_to_keep_in_mind_while_learning/

Rust Book: Invariants of the Type System
https://rustc-dev-guide.rust-lang.org/solve/invariants.html

Two Kinds of Invariants: Safety and Validity
https://www.ralfj.de/blog/2018/08/22/two-kinds-of-invariants.html

Sharing for a Lifetime
https://www.ralfj.de/blog/2018/01/31/sharing-for-a-lifetime.html